

- Să se scrie o aplicație pentru stocarea optimizată a datelor personale într-o bază de date de tip hashmap. Fiecare persoană este caracterizată de nume și înălțime. Se vor citi de la tastatură, în ordine, următoarele informații:
- un număr natural reprezentând capacitatea hashmap-ului (numărul de buckets = capacitatea);
 - un număr natural reprezentând numărul de persoane ce vor fi stocate în baza de date;
 - pentru fiecare persoană:
 - un șir de maxim 20 de caractere reprezentând numele;
 - un număr real reprezentând înălțimea, exprimată în metri;
 - un întreg reprezentând o comandă, în funcție de valoarea căreia se vor executa diferite funcționalități:
1. Afișarea datelor. Se vor afișa pe rânduri diferite informațiile citite de la tastatură, sub forma:
<nume> <inaltime>
 2. Creare hashmap. Se vor introduce elementele citite într-un hashmap, folosind metoda open addressing pentru gestionarea coliziunilor. În caz de coliziune pe un anumit index, se avansează la următorul index până se ajunge la prima poziție liberă din hashmap, iar elementul este introdus pe această poziție. Funcția hash utilizată va face următoarea procesare: se calculează suma codurilor ASCII ale numelui persoanei, aceasta se înmulțește cu înălțimea, iar partea întreagă a rezultatului este împărțită modulo la capacitatea hashmap-ului pentru a obține indexul bucket-ului final. Să se afișeze elementele hashmap-ului sub forma:

<index_bucket>: <nume>
 3. Parcurgere hashmap. Se vor parcurge elementele din baza de date în ordinea în care au fost citite de la tastatură și se va afișa pe ecran numărul total de comparații (se consideră comparație fiecare apel al funcției strcmp) necesar găsirii fiecărui element în hashmap sub forma:
<nume> <numar comparatii>
 4. Fast access-driven rehashing. Se citește de la tastatură o valoare întreagă reprezentând un prag de comparații. Se creează un hashmap din valorile citite de la tastatură cu capacitate egală cu numărul total de elemente citite. Ulterior, se face rehashing (crearea unui nou hashmap folosind funcția hash adaptată la noua capacitate) bazei de date în mod iterativ, la fiecare pas incrementând capacitatea cu o unitate, până când numărul total de comparații necesare parcurgerii întregului hashmap (conform subpct. 3) este mai mic decât pragul citit de la tastatură. Să se afișeze valoarea capacității dorite și apoi să se afișeze elementele hashmap-ului conform subpct 2.

Test #1

Input	Output
20 15 Sandra 1.67 George 1.83 Maria 1.64 Mariana 1.59 Maricica 1.71 Airwrecka 1.73 Mihaela 1.61 Francois 1.80 Americaniqua 1.68	Sandra 1.67 George 1.83 Maria 1.64 Mariana 1.59 Maricica 1.71 Airwrecka 1.73 Mihaela 1.61 Francois 1.80 Americaniqua 1.68 Darius 1.93 Dumitru 1.77 Vladimir 1.68 Boris 1.63 Xenia 1.58 Oana 1.62

Darius 1.93 Dumitru 1.77 Vladimir 1.68 Boris 1.63 Xenia 1.58 Oana 1.62 1	
Explicație: se citesc capacitatea (20) și numărul de persoane (15), după care se citesc datele celor 15 persoane. Comanda 1 corespunde afișării pe ecran a datelor citite.	

Test #2

Input	Output
20 15 Sandra 1.67 George 1.83 Maria 1.64 Mariana 1.59 Maricica 1.71 Airwrecka 1.73 Mihaela 1.61 Francois 1.80 Americaniqua 1.68 Darius 1.93 Dumitru 1.77 Vladimir 1.68 Boris 1.63 Xenia 1.58 Oana 1.62 2	0: Dumitru 1: Oana 2: 3: Sandra 4: Maria 5: Vladimir 6: 7: 8: Mariana 9: Mihaela 10: Americaniqua 11: Darius 12: Boris 13: Airwrecka 14: Xenia 15: 16: Maricica 17: Francois 18: 19: George
Explicație: se citesc capacitatea (20) și numărul de persoane (15), după care se citesc datele celor 15 persoane. Comanda 2 corespunde creării hashmap-ului folosind metoda open addressing pentru gestionarea coliziunilor. Exemplu coliziune: hash Sandra: $(83 (S) + 97 (a) + 110 (n) + 100 (d) + 114 (r) + 97 (a)) * 1.67 = 1003.67$; $1003 \% 20 = 3$ hash Maria: $(77 (M) + 97 (a) + 114 (r) + 105 (i) + 97 (a)) * 1.64 = 803.6$; $803 \% 20 = 3$ Prin urmare, Maria va fi mutată la următoarea poziție disponibilă, adică index = 4.	

Test #3

Input	Output
20 15 Sandra 1.67 George 1.83 Maria 1.64 Mariana 1.59 Maricica 1.71 Airwrecka 1.73 Mihaela 1.61 Francois 1.80 Americaniqua 1.68 Darius 1.93 Dumitru 1.77 Vladimir 1.68 Boris 1.63 Xenia 1.58 Oana 1.62 3	Sandra 1 George 1 Maria 2 Mariana 1 Maricica 1 Airwrecka 1 Mihaela 1 Francois 1 Americaniqua 2 Darius 4 Dumitru 1 Vladimir 2 Boris 1 Xenia 4 Oana 2
Explicație: se citesc capacitatea (20) și numărul de persoane (15), după care se citesc datele celor 15 persoane. Comanda 3 corespunde aflării numărului de comparații necesare pentru a accesa elementele căutate. După calcularea hash-ului se face prima verificare/comparație. Dacă nu au existat coliziuni, atunci s-a rulat doar o singură comparație. Dacă au existat coliziuni, atunci numărul de comparații crește cu 1 pentru fiecare poziție cu care se face deplasarea în momentul inserției în hashmap.	

Test #1

Input	Output
20 15 Sandra 1.67 George 1.83 Maria 1.64 Mariana 1.59 Maricica 1.71 Airwrecka	33 0: Darius 1: Dumitru 2: 3: Maricica 4: 5: 6: 7: Boris 8: 9: Airwrecka 10: George 11: Maria

1.73	12:
Mihaela	13: Sandra
1.61	14:
Francois	15:
1.80	16:
Americaniqua	17:
1.68	18:
Darius	19: Mariana
1.93	20: Mihaela
Dumitru	21:
1.77	22:
Vladimir	23: Americaniqua
1.68	24:
Boris	25: Francois
1.63	26: Oana
Xenia	27:
1.58	28:
Oana	29:
1.62	30:
4	31: Vladimir
16	32: Xenia

Explicație: se citesc capacitatea (20) și numărul de persoane (15), după care se citesc datele celor 15 persoane. Comanda 4 corespunde găsirii celei mai mici valori a capacității pentru care numărul total de comparații pentru găsierea tuturor elementelor din baza de date este mai mic decât pragul citit (16). Sumarea numerelor corespunzătoare tuturor comparațiilor conform subpct. 3, în funcție de capacitate are următoarele valori:

```
Capacitate:15 ---> comparatii:38
Capacitate:16 ---> comparatii:45
Capacitate:17 ---> comparatii:29
Capacitate:18 ---> comparatii:22
Capacitate:19 ---> comparatii:27
Capacitate:20 ---> comparatii:25
Capacitate:21 ---> comparatii:37
Capacitate:22 ---> comparatii:24
Capacitate:23 ---> comparatii:32
Capacitate:24 ---> comparatii:21
Capacitate:25 ---> comparatii:18
Capacitate:26 ---> comparatii:20
Capacitate:27 ---> comparatii:20
Capacitate:28 ---> comparatii:21
Capacitate:29 ---> comparatii:20
Capacitate:30 ---> comparatii:17
Capacitate:31 ---> comparatii:22
Capacitate:32 ---> comparatii:21
Capacitate:33 ---> comparatii:16
```

Prima valoare a capacității pentru care numărul de comparații scade la maxim 16 este 33. Se afișează starea hashmap-ului pentru această valoare a capacității.